

Cypress

Interview Questions & Answers

100 Questions — Easy • Hard • Scenario-Based

35	30	35	100
Easy	Hard	Scenario	Total

PREPARED BY

Rahul Dange
Senior Lead Quality Engineer
April 2026

Table of Contents

1. Easy Questions (Q1 – Q35)

35 fundamental Cypress concepts

2. Hard Questions (Q36 – Q65)

30 advanced & internals questions

3. Scenario-Based Questions (Q66 – Q100)

35 real-world test scenarios

4. Tips & Best Practices

Senior-level guidance for production test suites

5. Common Mistakes to Avoid

Top 10 anti-patterns that hurt test quality

6. Must-Know Plugins

Curated plugin reference for advanced testing

7. Cypress Cheat Sheet

Quick-reference command & assertion guide

8. Interview Prep Tracker

Checklist to track your revision progress

1. Easy Questions

Q1.

What is Cypress and how is it different from Selenium?

Cypress is a modern, JavaScript-based end-to-end testing framework that runs directly in the browser. Unlike Selenium, which uses WebDriver to communicate with the browser externally, Cypress runs inside the browser in the same event loop as the application. This gives it native access to DOM events, network requests, and application code. Cypress also provides automatic waiting, real-time reloading, and built-in screenshots and video recording.

Q2.

What languages and frameworks does Cypress support?

Cypress is JavaScript/TypeScript-based and works natively with React, Vue, Angular, Next.js, Svelte, and more. It uses Mocha as the test runner and Chai for assertions, with jQuery-style DOM traversal built in.

Q3.

How do you install Cypress?

Install via npm or yarn:

```
npm install cypress --save-dev
# Then open the Cypress Test Runner:
npx cypress open
```

Q4.

What is the Cypress Test Runner?

The Cypress Test Runner is a visual interface showing test files, running them interactively, displaying a command log of every action, and rendering the app in a browser side-by-side. It provides time-travel debugging where you can hover any command to see the DOM state at that point.

Q5.

What is `cy.visit()` used for?

`cy.visit()` navigates the browser to a specified URL and waits for the page to fully load before proceeding.

```
cy.visit('https://example.com')
cy.visit('/dashboard') // relative to baseUrl
```

Q6.

How do you select DOM elements in Cypress?

Use `cy.get()` with CSS selectors, `cy.contains()` for text, or `cy.find()` within a parent:

```
cy.get('.btn-submit')
cy.get('[data-cy="login-btn"]')
cy.contains('Submit')
cy.get('form').find('input[name="email"]')
```

Q7.

What is the recommended way to select elements for testing?

Cypress recommends using custom data attributes like `data-cy`, `data-test`, or `data-testid`. These are decoupled from styling and unaffected by CSS/JS refactoring. Example: then select with `cy.get('[data-cy="submit"]')`.

Q8.

How do you type into an input field in Cypress?

Use `.type()` on a focused element. Special keys like `{enter}`, `{tab}`, `{backspace}` can be included in the string.

```
cy.get('input[name="email"]').type('user@example.com')
cy.get('input[type="password"]').type('mypassword{enter}')
```

Q9.

How do you click a button in Cypress?

Use the `.click()` command:

```
cy.get('button[type="submit"]').click()
cy.contains('Login').click()
cy.get('.menu-item').first().click()
```

Q10.

What are Cypress assertions and what libraries power them?

Cypress bundles Chai, Sinon, and jQuery Chai for assertions. Use `.should()` and `.and()` to chain:

```
cy.get('h1').should('be.visible').and('contain.text', 'Welcome')
cy.get('[data-cy="count"]').should('have.text', '5')
cy.url().should('include', '/dashboard')
```

Q11.

What is the difference between `.should()` and `.expect()`?

`.should()` is a Cypress command that retries until the assertion passes or times out — async and chainable. `.expect()` is a synchronous Chai assertion used inside `.then()` callbacks. Prefer `.should()` for Cypress-managed elements.

Q12.

What is `cy.intercept()`?

`cy.intercept()` intercepts and optionally stubs HTTP requests, letting you control network responses in tests.

```
cy.intercept('GET', '/api/users', { fixture: 'users.json' }).as('getUsers')
cy.wait('@getUsers')
cy.get('[data-cy="user-list"]').should('have.length', 3)
```

Q13.

What are fixtures in Cypress?

Fixtures are static JSON files in `cypress/fixtures` that simulate API responses so tests don't depend on real backend data. Use `cy.fixture('filename')` to load one and `cy.intercept()` to use it as a response stub.

Q14.

How do you handle alerts and confirm dialogs in Cypress?

Cypress automatically dismisses `window.alert()` and `window.confirm()`. To stub `confirm` to return `false`:

```
cy.on('window:confirm', () => false)
cy.get('[data-cy="delete"]').click()
// confirm returns false, deletion is cancelled
```

Q15.

What is `beforeEach` in Cypress?

`beforeEach` is a Mocha lifecycle hook that runs before each test in a `describe` block — used for setting up preconditions like visiting a page or logging in.

```
beforeEach(() => {
  cy.visit('/login')
  cy.get('[data-cy="email"]').type('admin@test.com')
  cy.get('[data-cy="submit"]').click()
})
```

Q16.

How do you run Cypress in headless mode (CI)?

Use the `run` command instead of `open`:

```
npx cypress run
npx cypress run --browser chrome
npx cypress run --spec 'cypress/e2e/login.cy.js'
```

Q17.

What file structure does Cypress use?

Typical structure: `cypress/e2e/` for test files, `cypress/fixtures/` for mock data, `cypress/support/` for custom commands and setup files (`e2e.js` runs before every spec), and `cypress.config.js` at the root for configuration.

Q18.

What is `cy.wrap()`?

`cy.wrap()` wraps a value so you can use Cypress commands and assertions on it:

```
const user = { name: 'Alice' }
cy.wrap(user).its('name').should('eq', 'Alice')
```

Q19.

How do you clear an input field in Cypress?

Use `.clear()` before `.type()` to remove existing content:

```
cy.get('input[name="username"]').clear().type('newuser')
```

Q20.

How do you take screenshots and record videos in Cypress?

Screenshots are taken automatically on test failure and can be triggered manually with `cy.screenshot()`.

Videos are recorded in headless mode and saved to `cypress/videos`. Both are configured in `cypress.config.js`.

Q21.

What is `cy.wait()` and when should you use it?

`cy.wait()` with a number pauses execution — generally discouraged. The preferred use is waiting for aliased network intercepts:

```
cy.intercept('POST', '/api/login').as('loginReq')
cy.get('[data-cy="submit"]').click()
cy.wait('@loginReq').its('response.statusCode').should('eq', 200)
```

Q22.

What is the `cypress.config.js` file?

The main configuration file for Cypress:

```
module.exports = defineConfig({
  e2e: {
    baseUrl: 'http://localhost:3000',
    viewportWidth: 1280,
    defaultCommandTimeout: 6000,
    video: false
  }
})
```

Q23.

How do you create custom commands in Cypress?

Add them to `cypress/support/commands.js`:

```
Cypress.Commands.add('login', (email, password) => {
  cy.get('[data-cy="email"]').type(email)
  cy.get('[data-cy="password"]').type(password)
  cy.get('[data-cy="submit"]').click()
})
// Usage:
cy.login('user@test.com', 'password123')
```

Q24.

What does `cy.contains()` do?

`cy.contains()` finds elements containing specific text:

```
cy.contains('Submit') // any element
cy.contains('button', 'Submit') // specific tag
cy.get('.nav').contains('Home') // scoped
```

Q25.

What is `cy.reload()` used for?

`cy.reload()` reloads the current page:

```
cy.reload() // soft reload
cy.reload(true) // hard reload (bypasses cache)
```

Q26.

What is `cy.request()` used for?

`cy.request()` makes direct HTTP requests from within tests — useful for API testing, seeding data, or bypassing the UI for setup.

```
cy.request('POST', '/api/users', { name: 'Bob', email: 'bob@test.com' })
  .its('status').should('eq', 201)
```

Q27.

What is the default timeout in Cypress?

The `defaultCommandTimeout` is 4000ms. Configure it globally in `cypress.config.js` or override per command with `{ timeout: N }`. `cy.visit()` has its own `pageLoadTimeout` defaulting to 60 seconds.

Q28.

How do you check if an element exists or doesn't exist?

Use `.should('exist')` / `.should('not.exist')` or `.should('be.visible')` / `.should('not.be.visible')`.

```
cy.get('[data-cy="modal"]').should('exist')
cy.get('[data-cy="modal"]').should('not.exist')
cy.get('[data-cy="spinner"]').should('not.be.visible')
```

Q29.

What is the difference between `'not.exist'` and `'not.be.visible'`?

`'not.exist'` asserts the element is completely absent from the DOM. `'not.be.visible'` asserts the element exists but is hidden (`display:none` or `visibility:hidden`). Use `'not.exist'` after a delete; use `'not.be.visible'` for toggled visibility.

Q30.

How do you check a checkbox in Cypress?

Use `.check()` and `.unchecked()`:

```
cy.get('[data-cy="terms"]').check()
cy.get('[data-cy="terms"]').should('be.checked')
cy.get('input[type="checkbox"]').check(['option1', 'option3'])
```

Q31.

How do you select an option from a dropdown?

Use `.select()` with the option text or value:

```
cy.get('select[name="country"]').select('India')
cy.get('select[name="country"]').select('IN') // by value
cy.get('select[name="country"]').should('have.value', 'IN')
```

Q32.

What is `cy.exec()` used for?

`cy.exec()` runs a shell command in the OS from within a test:

```
cy.exec('node scripts/seed-db.js')
cy.exec('rm -rf cypress/downloads')
```

Q33.

How do you hover over an element in Cypress?

Cypress has no native `.hover()` since CSS `:hover` is a browser state. Use `.trigger('mouseover')` or `.trigger('mouseenter')`:

```
cy.get('[data-cy="menu-item"]').trigger('mouseover')
cy.get('[data-cy="dropdown"]').should('be.visible')
```

Q34.

What is `.invoke()` used for in Cypress?

`cy.invoke()` calls a function on the yielded subject:

```
cy.get('input').invoke('val') // get value
cy.get('[data-cy="price"]').invoke('text').then(parseFloat)
cy.window().invoke('scrollTo', 0, 500) // scroll
```

Q35.

What is `.its()` used for?

`cy.its()` retrieves a property from the yielded object:

```
cy.request('/api/user').its('body.name').should('eq', 'Alice')
cy.get('[data-cy="item"]').its('length').should('eq', 5)
```

2. Hard Questions

Q36.

How does Cypress handle asynchronous code and the event loop?

Cypress commands are not Promises — they are enqueued synchronously and executed asynchronously in order. Each command yields its subject to the next. Cypress manages its own command queue separate from the JavaScript Promise chain, using a run-loop that drains the queue and automatically retries assertions until they pass or timeout.

Q37.

Why can't you use `async/await` with Cypress commands directly?

Cypress commands are synchronous enqueueers, not Promises. `cy.get()` returns a Cypress chainable, not a Promise. To work with values use `.then()`:

```
// WRONG:
const el = await cy.get('.title')

// CORRECT:
cy.get('.title').then(($el) => {
  const text = $el.text()
  expect(text).to.include('Dashboard')
})
```

Q38.

What is the difference between `cy.stub()` and `cy.spy()`?

`cy.spy()` wraps a function and records calls but still executes the original. `cy.stub()` replaces a function entirely with a fake you control.

```
const stub = cy.stub(console, 'log')
stub.withArgs('clicked').as('logCalled')
```

```
cy.get('[data-cy="btn"]').click()
cy.get('@logCalled').should('have.been.calledOnce')
```

Q39.

How do you handle cross-origin iframes in Cypress?

Cypress 9.6+ added `cy.origin()` to handle cross-origin navigation:

```
cy.visit('https://app.mysite.com')
cy.get('[data-cy="sso-link"]').click()
cy.origin('https://auth.provider.com', () => {
  cy.get('#username').type('user@test.com')
  cy.get('#submit').click()
})
```

Q40.

What is `cy.task()` and when would you use it?

`cy.task()` runs code in the Node.js process (not the browser) — for operations like database seeding, reading files, or sending emails in test setups.

```
// cypress.config.js
on('task', {
  seedDatabase() { return db.seed(); return null }
})
// In test:
cy.task('seedDatabase')
```

Q41.

How do you test file uploads in Cypress?

Use `.selectFile()` (Cypress 9.3+):

```
cy.get('input[type="file"]').selectFile('cypress/fixtures/sample.pdf')
// Drag and drop:
cy.get('[data-cy="dropzone"]').selectFile(
  'cypress/fixtures/image.png',
  { action: 'drag-drop' }
)
```

Q42.

How do you test file downloads in Cypress?

Cypress can't interact with the native download dialog. Use `cy.readFile()` to verify the file appeared:

```
cy.get('[data-cy="download-btn"]').click()
cy.readFile('cypress/downloads/report.csv')
  .should('contain', 'Name,Email')
```

Q43.

How do you set and read cookies in Cypress?

Use `setCookie`, `getCookie`, and `clearCookies`:

```
cy.setCookie('session_token', 'abc123')
cy.getCookie('session_token')
  .should('have.property', 'value', 'abc123')
cy.clearCookies()
```

Q44.

What is `cy.session()` and why is it better than using `beforeEach` login?

`cy.session()` caches and restores browser session state (cookies, `localStorage`, `sessionStorage`) across tests. It only re-runs setup if the session is invalid, making test suites significantly faster.

```
Cypress.Commands.add('loginBySession', (user) => {
  cy.session(user, () => {
    cy.visit('/login')
    cy.get('[data-cy="email"]').type(user.email)
    cy.get('[data-cy="submit"]').click()
    cy.url().should('include', '/dashboard')
  })
})
```

Q45.

How do you perform API-only login in Cypress for speed?

Use `cy.request()` to hit the login API and set the session token directly, skipping the UI:

```
cy.request({
  method: 'POST',
  url: '/api/auth/login',
  body: { email: 'user@test.com', password: 'secret' }
}).then(({ body }) => {
  window.localStorage.setItem('authToken', body.token)
  cy.setCookie('session', body.sessionId)
})
cy.visit('/dashboard')
```

Q46.

How does Cypress retry-ability work?

Cypress automatically retries most commands and assertions until they pass or timeout. Retrying happens only for queries (`.get`, `.find`, `.contains`) and assertions (`.should`, `.and`). Action commands (`.click`, `.type`) are not retried. Cypress re-runs the entire chain from the last query point.

Q47.

What is the Cypress network layer and how does `cy.intercept` differ from `cy.route`?

`cy.route()` was the old API (Cypress 4) that only worked with XHR and required `cy.server()`. `cy.intercept()` (Cypress 6+) handles both XHR and fetch, supports glob/regex patterns, can modify request bodies/headers, and doesn't require `cy.server()`. `cy.route()` is deprecated.

Q48.

How do you test `localStorage` and `sessionStorage`?

Access storage inside `cy.window()`:

```
cy.window().then((win) => {
  expect(win.localStorage.getItem('cart')).to.eq('[]')
  win.localStorage.setItem('featureFlag', 'true')
})
```

Q49.

How do you handle dynamic elements that appear after a delay?

Cypress automatically retries queries. Increase timeout for specific commands:

```
cy.get('[data-cy="spinner"]').should('not.exist')
cy.get('[data-cy="results"]').should('be.visible')
```

Q50.

How do you run tests in parallel in Cypress?

Parallel execution requires Cypress Cloud. Set up with `--parallel --record` flags in CI. Cypress splits spec files across machines based on historical run times. Without Cypress Cloud, manually split specs by configuring multiple CI jobs with different spec patterns.

Q51.

How do you write custom assertions in Cypress with Chai?

Add custom Chai assertions in `cypress/support/e2e.js`:

```
chai.Assertion.addMethod('withinRange', function(min, max) {
  const val = this._obj
  this.assert(
    val >= min && val <= max,
    'expected #{this} to be within ' + min + ' and ' + max
  )
})
// Usage:
cy.get('[data-cy="score"]').invoke('text')
  .then(Number).should('be.withinRange', 50, 100)
```

Q52.

What is the Cypress Module API and when is it used?

The Module API lets you control Cypress programmatically from a Node script — useful for advanced CI pipelines:

```
const cypress = require('cypress')
cypress.run({
  spec: 'cypress/e2e/checkout.cy.js',
  browser: 'chrome',
  headless: true
}).then(results => {
  if (results.totalFailed > 0) process.exit(1)
})
```

Q53.

How do you handle browser permissions (camera, geolocation) in Cypress?

Use browser launch options for Chrome permissions:

```
on('before:browser:launch', (browser, launchOptions) => {
  if (browser.name === 'chrome') {
    launchOptions.args.push('--use-fake-device-for-media-stream')
    launchOptions.args.push('--use-fake-ui-for-media-stream')
  }
  return launchOptions
})
```

Q54.

How do you test drag and drop in Cypress?

Use the @4tw/cypress-drag-drop plugin or trigger events manually:

```
// Plugin:
cy.get('[data-cy="draggable"]').drag('[data-cy="droppable"]')

// Manual:
cy.get('[data-cy="source"]')
  .trigger('mousedown', { button: 0 })
  .trigger('mousemove', { clientX: 400, clientY: 200 })
cy.get('[data-cy="target"]').trigger('mouseup', { force: true })
```

Q55.

How does Cypress handle shadow DOM?

Use the includeShadowDom option or .shadow():

```
// Global config:
includeShadowDom: true

// Per-command:
cy.get('[data-cy="web-component"]', { includeShadowDom: true })
  .find('[data-cy="inner-btn"]').click()

// Using .shadow():
cy.get('my-element').shadow().find('button').click()
```

Q56.

What is the difference between cy.get() and cy.find()?

cy.get() always queries from the document root. cy.find() queries within the DOM subtree of the previously yielded element. Use cy.find() for scoped selection within a parent to avoid unintended matches.

Q57.

How do you debug Cypress tests?

Several approaches: (1) cy.pause() to pause execution; (2) cy.debug() to log the current subject; (3) debugger; inside .then() callbacks; (4) time-travel debugger in the Cypress UI; (5) cy.log() for custom log entries.

```
cy.get('[data-cy="cart"]').debug()
cy.get('[data-cy="item"]').then(($el) => { debugger; })
```

Q58.

How do you handle multiple tabs or windows in Cypress?

Cypress intentionally doesn't support multiple tabs (single browser context). Remove/mock target='_blank', test links by asserting href, or use cy.request() to test linked URLs independently. For OAuth popups, stub window.open and simulate the callback.

Q59.

How do you configure retries in Cypress for flaky tests?

Configure globally or per-test:

```
// Global:
retries: { runMode: 2, openMode: 0 }

// Per-test:
```

```
it('flaky test', { retries: 3 }, () => {
  cy.visit('/unstable-page')
})
```

Q60.

How do you use `cy.intercept()` to modify a request body before it's sent?

Use a callback function in `cy.intercept()`:

```
cy.intercept('POST', '/api/orders', (req) => {
  req.body.discount = 10
  req.body.coupon = 'TEST10'
  req.continue() // let real request proceed
}).as('orderReq')
```

Q61.

What is `StaticResponse` in `cy.intercept()`?

`StaticResponse` is the object format for stubbing responses, supporting `statusCode`, `body`, `headers`, `delay`, and `throttleKbps`.

```
cy.intercept('GET', '/api/users', {
  statusCode: 200,
  body: [{ id: 1, name: 'Alice' }],
  headers: { 'x-custom': 'test' },
  delay: 500,
  throttleKbps: 64
})
```

Q62.

What is `testIsolation` in Cypress 12+ and what changed from earlier versions?

In Cypress 12+, `testIsolation: true` is default — Cypress clears all cookies, `localStorage`, and `sessionStorage` before each test. Previously state persisted between tests, causing test order dependencies and flakiness. You can disable isolation per describe block with `{ testIsolation: false }`.

Q63.

How do you test `WebSocket` connections in Cypress?

Cypress doesn't natively intercept `WebSocket` frames. Stub the `WebSocket` constructor:

```
cy.window().then((win) => {
  cy.stub(win, 'WebSocket').callsFake((url) => {
    return { send: cy.stub(), close: cy.stub(),
      addEventListener: cy.stub() }
  })
})
```

Q64.

How do you use `Cypress Component Testing`?

Component Testing mounts individual components in isolation without a running server:

```
// cypress/component/Button.cy.jsx
import Button from '../../src/Button'

it('calls onClick when clicked', () => {
  const onClick = cy.stub()
  cy.mount(Click me)
```

```
cy.get('button').click()
expect(onClick).to.have.been.calledOnce
})
```

Q65.

How do you measure and assert page performance metrics in Cypress?

Use the Performance API through `cy.window()`:

```
cy.window().then((win) => {
  const [nav] = win.performance.getEntriesByType('navigation')
  expect(nav.domContentLoadedEventEnd - nav.startTime)
    .to.be.lessThan(3000)
})
```

Q66.

How do you run Cypress tests with different environment configurations?

Use `--env` flag or `cypress.env.json` and access with `Cypress.env()`:

```
// Run with env vars:
npx cypress run --env apiUrl=https://staging.api.com

// Access in test:
const url = Cypress.env('apiUrl')
cy.request(`${url}/health`).its('status').should('eq', 200)
```

Q67.

What is the Cypress grep plugin and how does it help?

The `@cypress/grep` plugin lets you run tests matching a specific keyword or tag:

```
it('smoke: login works', { tags: '@smoke' }, () => { ... })

// Run only smoke tests:
npx cypress run --env grepTags=@smoke

// Run by title:
npx cypress run --env grep="login"
```

3. Scenario-Based Questions

Q68.

Scenario: A login page redirects to /dashboard on success. Write a complete Cypress test.

Complete login flow test with network interception:

```
describe('Login flow', () => {
  it('logs in and redirects to dashboard', () => {
    cy.intercept('POST', '/api/auth/login').as('loginReq')
    cy.visit('/login')
    cy.get('[data-cy="email"]').type('user@example.com')
    cy.get('[data-cy="password"]').type('password123')
    cy.get('[data-cy="login-btn"]').click()
    cy.wait('@loginReq').its('response.statusCode').should('eq', 200)
    cy.url().should('include', '/dashboard')
    cy.get('[data-cy="welcome-msg"]').should('contain.text', 'Welcome')
  })
  it('shows error for invalid credentials', () => {
```

```

cy.visit('/login')
cy.get('[data-cy="email"]').type('wrong@example.com')
cy.get('[data-cy="password"]').type('wrongpass')
cy.get('[data-cy="login-btn"]').click()
cy.get('[data-cy="error-msg"]').should('be.visible')
.and('contain.text', 'Invalid credentials')
})
})

```

Q69.

Scenario: Test a shopping cart that adds and removes items, updating the total price.

Shopping cart test with fixture-based data:

```

it('adds item and verifies total updates', () => {
  cy.intercept('GET', '/api/products',
    { fixture: 'products.json' }).as('loadProducts')
  cy.visit('/shop')
  cy.wait('@loadProducts')
  cy.get('[data-cy="add-to-cart"]').first().click()
  cy.get('[data-cy="cart-count"]').should('have.text', '1')
  cy.get('[data-cy="remove-item"]').click()
  cy.get('[data-cy="cart-empty-msg"]').should('be.visible')
})

```

Q70.

Scenario: A form has conditional fields — Billing address appears only when a checkbox is unchecked.

Conditional field visibility test:

```

it('shows billing address when unchecked', () => {
  cy.visit('/checkout')
  cy.get('[data-cy="same-as-shipping"]').should('be.checked')
  cy.get('[data-cy="billing-address"]').should('not.be.visible')
  cy.get('[data-cy="same-as-shipping"]').uncheck()
  cy.get('[data-cy="billing-address"]').should('be.visible')
  cy.get('[data-cy="billing-street"]').type('123 Main St')
})

```

Q71.

Scenario: An API call on page load fails with a 500 error. How do you test the error state?

Test error UI when API returns 500:

```

it('shows error banner when API returns 500', () => {
  cy.intercept('GET', '/api/dashboard-data', {
    statusCode: 500,
    body: { message: 'Internal Server Error' }
  }).as('dashboardData')
  cy.visit('/dashboard')
  cy.wait('@dashboardData')
  cy.get('[data-cy="error-banner"]').should('be.visible')
  .and('contain.text', 'Something went wrong')
  cy.get('[data-cy="retry-btn"]').should('be.visible')
})

```

Q72.

Scenario: Test a paginated table that loads more results on clicking Next.

Pagination navigation test:

```
it('navigates pages and shows correct records', () => {
  cy.intercept('GET', '/api/users?page=1',
    { fixture: 'users-page1.json' }).as('page1')
  cy.intercept('GET', '/api/users?page=2',
    { fixture: 'users-page2.json' }).as('page2')
  cy.visit('/users')
  cy.wait('@page1')
  cy.get('[data-cy="user-row"]').should('have.length', 10)
  cy.get('[data-cy="next-btn"]').click()
  cy.wait('@page2')
  cy.get('[data-cy="page-info"]').should('contain.text', 'Page 2 of 5')
})
```

Q73.

Scenario: Test that a user cannot submit a form with an empty required field.

Form validation test:

```
it('prevents submission with empty required fields', () => {
  cy.visit('/register')
  cy.get('[data-cy="submit-btn"]').click()
  cy.get('[data-cy="email-error"]').should('be.visible')
    .and('contain.text', 'Email is required')
  cy.url().should('not.include', '/success')
})
```

Q74.

Scenario: Write a test for a search feature with debounce — results appear 500ms after typing.

Debounced search test:

```
it('shows search results after debounce delay', () => {
  cy.intercept('GET', '/api/search*',
    { fixture: 'search-results.json' }).as('searchReq')
  cy.visit('/search')
  cy.get('[data-cy="search-input"]').type('cypress')
  cy.wait('@searchReq')
  cy.get('[data-cy="result-item"]').should('have.length.greaterThan', 0)
    .first().should('contain.text', 'Cypress')
})
```

Q75.

Scenario: Test a multi-step wizard form with 3 steps and validation at each step.

Multi-step wizard form test:

```
it('completes all 3 steps', () => {
  cy.visit('/wizard')
  cy.get('[data-cy="first-name"]').type('Alice')
  cy.get('[data-cy="next"]').click()
  cy.get('[data-cy="step-indicator"]').should('contain.text', 'Step 2')
  cy.get('[data-cy="email"]').type('alice@test.com')
```

```

cy.get('[data-cy="next"]').click()
cy.get('[data-cy="confirm-checkbox"]').check()
cy.get('[data-cy="submit"]').click()
cy.get('[data-cy="success-msg"]').should('be.visible')
})

```

Q76.

Scenario: Your test is flaky — sometimes passes, sometimes fails on `.should('be.visible')`. How do you fix it?

1. Add `cy.intercept()` for related API calls and `cy.wait()` for them. 2. Check if another element overlaps it — use `.should('exist')` first. 3. Increase timeout on that specific command. 4. Disable CSS animations in `beforeEach`. 5. Use the Cypress time-travel debugger to inspect DOM state just before the assertion.

Q77.

Scenario: Test a date picker component — select a date and verify the form value updates.

Date picker interaction test:

```

it('selects a date from the date picker', () => {
  cy.visit('/booking')
  cy.get('[data-cy="date-picker"]').click()
  cy.get('[data-cy="next-month"]').click()
  cy.get('[data-cy="calendar-day"]').contains('15').click()
  cy.get('[data-cy="date-input"]').should('have.value', '2026-05-15')
  cy.get('[data-cy="calendar-popup"]').should('not.exist')
})

```

Q78.

Scenario: You need to test an infinite scroll list. How do you verify items load as you scroll?

Infinite scroll test:

```

it('loads more items on scroll', () => {
  cy.intercept('GET', '/api/feed?offset=0',
    { fixture: 'feed-0.json' }).as('page0')
  cy.intercept('GET', '/api/feed?offset=20',
    { fixture: 'feed-20.json' }).as('page1')
  cy.visit('/feed')
  cy.wait('@page0')
  cy.get('[data-cy="feed-item"]').should('have.length', 20)
  cy.get('[data-cy="feed-list"]').scrollTo('bottom')
  cy.wait('@page1')
  cy.get('[data-cy="feed-item"]').should('have.length', 40)
})

```

Q79.

Scenario: How do you test that an email was actually sent in an E2E test?

Use a mail testing service like Mailhog, Mailpit, or Mailtrap in your test environment. Trigger the email through the app, then use `cy.request()` to call the mail service REST API and assert the inbox contains the expected email with the correct recipient and subject.

Q80.

Scenario: Write a test that verifies a CSV export button triggers a correct download.

CSV download test via network interception:

```
it('exports data as CSV with correct content', () => {
  cy.intercept('GET', '/api/export/users.csv').as('csvDownload')
  cy.visit('/users')
  cy.get('[data-cy="export-csv"]').click()
  cy.wait('@csvDownload').then((interception) => {
    expect(interception.response.statusCode).to.eq(200)
    expect(interception.response.headers['content-type'])
      .to.include('text/csv')
    expect(interception.response.body).to.include('Name,Email,Role')
  })
})
```

Q81.

Scenario: How do you test behavior that differs by user role (admin vs viewer)?

Role-based test structure using parameterized describe blocks:

```
const users = {
  admin: { email: 'admin@test.com', password: 'adminpass' },
  viewer: { email: 'viewer@test.com', password: 'viewpass' }
}

Object.entries(users).forEach(([role, user]) => {
  describe(`Role: ${role}`, () => {
    beforeEach(() => cy.loginBySession(user))
    if (role === 'admin') {
      it('sees admin controls', () => {
        cy.get('[data-cy="admin-panel"]').should('be.visible')
      })
    } else {
      it('does not see admin controls', () => {
        cy.get('[data-cy="admin-panel"]').should('not.exist')
      })
    }
  })
})
```

Q82.

Scenario: Test a real-time chat application where messages appear without page refresh.

Real-time messaging test:

```
it('sends and receives messages in real time', () => {
  cy.intercept('POST', '/api/messages').as('sendMsg')
  cy.visit('/chat')
  cy.get('[data-cy="message-input"]').type('Hello team!')
  cy.get('[data-cy="send-btn"]').click()
  cy.wait('@sendMsg').its('response.statusCode').should('eq', 201)
  cy.get('[data-cy="message-list"]', { timeout: 10000 })
    .should('contain.text', 'Hello team!')
})
```

Q83.

Scenario: You have 200 tests and the suite takes 45 minutes. How do you speed it up?

1. Enable `cy.session()` for login caching.
2. Use `cy.request()` for API-based setup instead of UI navigation.
3. Run in parallel on Cypress Cloud or split specs across CI machines.
4. Replace `cy.wait(ms)` with network intercepts.
5. Set `video: false` in CI.
6. Use `testIsolation: false` carefully for expensive shared setup.
7. Tag slow tests and run them separately.

Q84.

Scenario: Test a dropdown that loads options asynchronously from an API.

Async dropdown test:

```
it('loads and selects async dropdown options', () => {
  cy.intercept('GET', '/api/countries',
    { fixture: 'countries.json' }).as('getCountries')
  cy.visit('/profile')
  cy.get('[data-cy="country-select"]').click()
  cy.wait('@getCountries')
  cy.get('[data-cy="dropdown-option"]').contains('India').click()
  cy.get('[data-cy="country-select"]').should('contain.text', 'India')
  cy.get('[data-cy="state-select"]').should('be.visible')
})
```

Q85.

Scenario: Write a test that verifies only authorized users can access the /admin route.

Access control test for admin route:

```
it('redirects unauthenticated user to login', () => {
  cy.clearCookies()
  cy.visit('/admin')
  cy.url().should('include', '/login')
})

it('allows admin role to access /admin', () => {
  cy.loginBySession({ email: 'admin@test.com', password: 'pass' })
  cy.visit('/admin')
  cy.url().should('include', '/admin')
  cy.get('[data-cy="admin-heading"]').should('be.visible')
})
```

Q86.

Scenario: An image carousel auto-advances every 3 seconds. How do you test it without real timers?

Carousel test using `cy.clock()` and `cy.tick()`:

```
it('auto-advances carousel using cy.clock()', () => {
  cy.clock()
  cy.visit('/home')
  cy.get('[data-cy="carousel-slide"]').eq(0).should('have.class', 'active')
  cy.tick(3000)
  cy.get('[data-cy="carousel-slide"]').eq(1).should('have.class', 'active')
  cy.tick(3000)
  cy.get('[data-cy="carousel-slide"]').eq(2).should('have.class', 'active')
})
```

Q87.

Scenario: Test an autocomplete input that filters suggestions as you type.

Autocomplete filter test:

```
it('filters suggestions based on typed text', () => {
  cy.intercept('GET', '/api/autocomplete?q=cy*',
    { body: [{ id:1, name:'Cypress' }, { id:2, name:'Cyprus' }] }).as('suggest')
  cy.visit('/search')
  cy.get('[data-cy="autocomplete-input"]').type('cy')
  cy.wait('@suggest')
  cy.get('[data-cy="suggestion-item"]').should('have.length', 2)
  cy.get('[data-cy="suggestion-item"]').contains('Cypress').click()
  cy.get('[data-cy="autocomplete-input"]').should('have.value', 'Cypress')
})
```

Q88.

Scenario: Test a modal dialog — opens, displays correct data, and closes on cancel.

Modal dialog test:

```
it('opens modal with correct data and closes on cancel', () => {
  cy.intercept('GET', '/api/user/42',
    { fixture: 'user-detail.json' }).as('getUser')
  cy.visit('/users')
  cy.get('[data-cy="user-row"]').contains('Alice')
  .parent().find('[data-cy="view-btn"]').click()
  cy.wait('@getUser')
  cy.get('[data-cy="modal"]').should('be.visible')
  cy.get('[data-cy="modal-name"]').should('have.text', 'Alice Smith')
  cy.get('[data-cy="modal-cancel"]').click()
  cy.get('[data-cy="modal"]').should('not.exist')
})
```

Q89.

Scenario: Test that a button is disabled while a form is submitting and re-enabled after.

Submit button disable state test:

```
it('disables submit during form submission', () => {
  cy.intercept('POST', '/api/register').as('registerReq')
  cy.visit('/register')
  cy.get('[data-cy="email"]').type('test@test.com')
  cy.get('[data-cy="submit"]').click()
  cy.get('[data-cy="submit"]').should('be.disabled')
  cy.wait('@registerReq')
  cy.get('[data-cy="submit"]').should('not.be.disabled')
})
```

Q90.

Scenario: Test a feature flag — component renders differently based on a flag in localStorage.

Feature flag test via localStorage injection:

```
it('shows beta feature when flag is enabled', () => {
  cy.visit('/dashboard', {
    onBeforeLoad(win) {
```

```

win.localStorage.setItem('featureFlags',
JSON.stringify({ betaChart: true }))
}
})
cy.get('[data-cy="beta-chart"]').should('be.visible')
})

```

Q91.

Scenario: Verify a table is sorted correctly when clicking column headers.

Table sorting test using array comparison:

```

it('sorts table by name column ascending', () => {
cy.visit('/products')
cy.get('[data-cy="col-name"]').click()
cy.get('[data-cy="row-name"]').then(($cells) => {
const names = [...$cells].map(el => el.innerText)
const sorted = [...names].sort()
expect(names).to.deep.equal(sorted)
})
})

```

Q92.

Scenario: Write a reusable Cypress command that seeds a user via API and logs in using session caching.

Reusable API seed + session login command:

```

Cypress.Commands.add('createAndLogin', (role = 'user') => {
cy.session(role, () => {
cy.request('POST', '/api/test/seed-user', { role })
.then(({ body }) => {
cy.request('POST', '/api/auth/login', {
email: body.email, password: body.password
}).then(({ body: authBody }) => {
window.localStorage.setItem('token', authBody.token)
})
})
})
cy.visit('/')
})

```

Q93.

Scenario: How do you test a toast notification that appears briefly (2s) after an action?

Toast notification test using cy.clock():

```

it('shows success toast after saving', () => {
cy.clock()
cy.visit('/settings')
cy.get('[data-cy="display-name"]').clear().type('New Name')
cy.get('[data-cy="save-btn"]').click()
cy.get('[data-cy="toast"]').should('be.visible')
.and('contain.text', 'Saved successfully')
cy.tick(2500)
cy.get('[data-cy="toast"]').should('not.exist')
})

```

Q94.

Scenario: Test that the browser back button restores the previous page state correctly.

Browser back button + state persistence test:

```
it('back button restores search results', () => {
  cy.visit('/search')
  cy.get('[data-cy="filter-category"]').select('Electronics')
  cy.get('[data-cy="search-input"]').type('headphones')
  cy.get('[data-cy="search-btn"]').click()
  cy.get('[data-cy="result-item"]').first().click()
  cy.go('back')
  cy.get('[data-cy="filter-category"]').should('have.value', 'Electronics')
  cy.get('[data-cy="search-input"]').should('have.value', 'headphones')
})
```

Q95.

Scenario: Your CI pipeline shows 15 tests consistently failing in CI but passing locally. How do you debug?

Common causes and fixes: (1) CI machines are slower — increase defaultCommandTimeout and pageLoadTimeout. (2) Missing env vars in CI — check all Cypress.env() values. (3) Test order dependency — ensure no test relies on state from previous tests. (4) Different viewport in CI — set explicitly in cypress.config.js. (5) No CSS animation disabling — add animation:none override in beforeEach. (6) Third-party network flakiness — stub all external APIs in CI. (7) Font/render differences — configure threshold in visual testing plugins.

4. Tips & Best Practices

1. Always use data-cy attributes for selectors

Never rely on CSS class names or XPath. Classes change during refactoring. data-cy attributes are test-only and stable. Set a lint rule to enforce their usage across your team.

```
cy.get('[data-cy="submit-btn"]').click()
```

2. Use cy.session() for all authentication

Logging in through the UI in every beforeEach is the #1 cause of slow test suites. cy.session() caches cookies, localStorage, and sessionStorage and only re-runs login when the session expires. A 200-test suite can go from 45 minutes to under 15.

```
cy.session('admin', () => { cy.request('POST', '/api/login', creds) })
```

3. Intercept all external API calls

Tests that depend on real network responses are flaky. Stub every API call using cy.intercept() with fixtures. This also lets you test error states (500s, 401s, timeouts) that would be hard to reproduce against a real server.

```
cy.intercept('GET', '/api/data', { fixture: 'data.json' }).as('getData')
```

4. Prefer cy.request() for test setup and teardown

Creating test data through the UI is slow and brittle. Use cy.request() to call your backend API to seed users, create records, and clean up after tests. Reserve the UI for what you're actually testing.

```
cy.request('POST', '/api/test/seed', { role: 'admin' })
```

5. Never use cy.wait(milliseconds)

Hard-coded waits make tests slow and are still flaky. Replace every cy.wait(2000) with a cy.intercept() + cy.wait('@alias') combination. This waits exactly as long as needed and no longer.

```
// Bad: cy.wait(2000)
// Good:
cy.intercept('GET', '/api/profile').as('profile')
cy.wait('@profile')
```

6. Organise tests by user journey, not by page

Group tests as a user would experience them: Login Flow, Checkout Flow, Admin Workflow. This makes test intent clear and helps identify gaps. Use describe() blocks to represent each journey.

```
describe('Checkout Journey', () => {
  it('adds item to cart')
  it('enters payment details')
  it('confirms order')
})
```

7. Run smoke tests on every PR, regression nightly

Tag tests with @smoke and @regression using the @cypress/grep plugin. Run only smoke tests on every pull request (fast, ~5 min) and the full suite in nightly CI. This gives fast feedback without blocking deployments.

```
it('critical login', { tags: '@smoke' }, () => { ... })
```

8. Write tests at the right level of the pyramid

Unit tests for pure logic, component tests (Cypress CT) for UI components in isolation, and E2E tests only for critical user journeys. Avoid duplicating coverage across levels — keep E2E tests focused on integration points.

9. Disable animations in test environments

CSS animations and transitions cause timing issues and flaky `.should('be.visible')` assertions. Add a global CSS rule that disables animations in your test environment or inject it in `cypress/support/e2e.js`.

```
// support/e2e.js
cy.document().then((doc) => {
  const style = doc.createElement('style')
  style.innerHTML = '* { animation: none !important; transition: none !important; }'
  doc.head.appendChild(style)
})
```

10. Use meaningful test descriptions

Write test names as user stories: 'it should show an error message when the email is invalid' not 'it tests email'. Good test names are living documentation — they tell the next engineer exactly what behaviour is being verified.

5. Common Mistakes to Avoid

1. Using `cy.wait(ms)` everywhere

Hard-coded waits are the most common Cypress anti-pattern. They make tests slow and are still flaky if the app is slower than expected. Always replace with `cy.intercept()` aliases or retry-safe assertions.

```
// WRONG: cy.wait(3000)
// RIGHT: cy.wait('@apiAlias')
```

2. Selecting elements by CSS class or tag only

Classes and HTML tags change during styling updates, breaking tests silently. Always use `data-cy`, `data-test`, or `data-testid` attributes. If you can't modify the HTML, use `aria-label` or stable role attributes as fallbacks.

```
// WRONG: cy.get('.btn.primary').click()
// RIGHT: cy.get('[data-cy="submit"]').click()
```

3. Logging in through the UI in every test

UI login in `beforeEach` is the primary reason test suites are slow. Switch to `cy.session()` or API-based login via `cy.request()` for all tests that aren't specifically testing the login page itself.

4. Testing implementation details instead of behaviour

Avoid asserting on internal state like Redux store values or class names that encode state. Test what the user sees and can do — visible text, enabled/disabled buttons, URLs — not how the code works internally.

5. Writing tests that depend on each other

Each test must be fully independent and runnable in any order. Shared state (leftover cookies, data from a previous test) causes mysterious failures. Use `testIsolation: true` (Cypress 12+ default) and reset state in `beforeEach`.

6. Ignoring `cy.intercept()` for third-party APIs

Tests that call real third-party services (payment gateways, analytics, email APIs) are flaky by nature. Always stub external dependencies — this also prevents you from polluting production data during test runs.

```
cy.intercept('POST', 'https://api.stripe.com/**', { fixture: 'stripe-success.json' })
```

7. Not using `.as()` aliases for intercepts

Without aliases, you can't assert on the network call after it happens. Always alias your intercepts and use `cy.wait('@alias')` to verify the request was made with the correct payload.

```
cy.intercept('POST', '/api/order').as('createOrder')
cy.wait('@createOrder').its('request.body.items').should('have.length', 2)
```

8. Using `{ force: true }` to bypass visibility checks

`cy.click({ force: true })` skips Cypress's actionability checks, hiding real bugs where elements are covered or not user-interactable. Only use `force` when you have a deliberate reason — investigate the root cause first.

9. Skipping error state and edge case tests

Most QA covers the happy path. Senior-level testing includes: empty states, 404/500 API responses, network timeouts, concurrent requests, invalid inputs, and permission-denied scenarios. These catch the most critical bugs.

10. Not using Cypress Cloud for flake detection

Cypress Cloud records test runs and automatically flags tests that sometimes pass and sometimes fail. Without it, you're debugging flaky tests blindly. Enable recording with `--record` to get flake analytics, parallel runs, and run history.

```
npx cypress run --record --key --parallel
```

6. Must-Know Plugins

@cypress/grep

Run a subset of tests by title keyword or custom tag without touching spec files. Essential for separating smoke, regression, and slow tests in CI.

```
npm install @cypress/grep --save-dev
npx cypress run --env grepTags=@smoke
```

cypress-axe

Automated accessibility testing using the axe-core engine. Inject it into any page and assert zero violations — integrates naturally into existing E2E tests.

```
npm install cypress-axe axe-core --save-dev
cy.injectAxe()
cy.checkA11y(null, null, (violations) => {
  violations.forEach(v => cy.log(v.description))
})
```

cypress-real-events

Simulates real browser events (hover, drag, keyboard shortcuts) using Chrome DevTools Protocol instead of synthetic DOM events. Necessary when `.trigger()` is not enough.

```
npm install cypress-real-events --save-dev
cy.get('[data-cy="menu-item"]').realHover()
cy.get('[data-cy="dropdown"]').should('be.visible')
```

@4tw/cypress-drag-drop

Reliable drag-and-drop support for HTML5 drag events and sortable lists. Much simpler API than manually triggering mousedown/mousemove/mouseup events.

```
npm install @4tw/cypress-drag-drop --save-dev
cy.get('[data-cy="card"]').drag('[data-cy="target-column"]')
```

cypress-iframe

Adds `.frameLoaded()` and `.iframe()` commands for interacting with same-origin iframes that are common in embedded third-party widgets.

```
npm install cypress-iframe --save-dev
cy.frameLoaded('[data-cy="iframe"]')
cy.iframe().find('button').click()
```

cypress-lighthouse

Run Google Lighthouse performance, SEO, accessibility, and best-practice audits from within a Cypress test. Assert Core Web Vitals scores as part of your CI pipeline.

```
npm install cypress-lighthouse --save-dev
cy.lighthouse({ performance: 85, accessibility: 90, seo: 80 })
```

@badeball/cypress-cucumber-preprocessor

Write Cypress tests in Gherkin (Given/When/Then) syntax for BDD-style test specs. Useful when business stakeholders need to read and contribute to test scenarios.

```
npm install @badeball/cypress-cucumber-preprocessor --save-dev
Given('I am on the login page', () => cy.visit('/login'))
When('I enter valid credentials', () => { ... })
Then('I should see the dashboard', () => { ... })
```

cypress-mailhog / cypress-mailtrap

Interact with a local Mailhog server or Mailtrap inbox to assert that emails were sent with the correct subject, recipient, and body content — essential for testing registration, password reset, and notification flows.

```
npm install cypress-mailhog --save-dev
cy.mhGetAllMails().should('have.length', 1)
cy.mhGetAllMails().its(0).its('Content.Headers.Subject.0')
  .should('eq', 'Welcome to the app!')
```

7. Cypress Cheat Sheet

Navigating

```
cy.visit('/path') // navigate
```

```
cy.go('back') / cy.go('forward')
```

```
cy.reload() / cy.reload(true) // hard reload  
cy.url().should('include', '/dashboard')
```

```
cy.title().should('eq', 'Home')
```

Selecting Elements

```
cy.get('[data-cy="btn"]') // by attribute
```

```
cy.contains('Submit') // by text  
cy.contains('button', 'Login') // tag + text  
cy.get('form').find('input') // scoped find  
cy.get('li').first() / .last()  
cy.get('li').eq(2) // by index
```

Actions

```
el.click() / .dblclick() / .rightclick()  
el.type('text{enter}') // type + special keys  
el.clear().type('new value')  
el.check() / .unchecked()  
el.select('optionText')  
el.scrollToView()  
el.trigger('mouseover')
```

```
el.selectFile('path/to/file') // upload
```

Assertions

```
.should('exist') / ('not.exist')  
.should('be.visible') / ('not.be.visible')  
.should('have.text', 'Hello')  
.should('contain.text', 'ello')
```

```
.should('have.value', 'input value')  
.should('have.length', 5)  
.should('have.class', 'active')  
.should('have.attr', 'href', '/home')  
.should('be.disabled') / ('be.enabled')  
.should('be.checked')
```

```
.and('include', 'substring') // chain
```

Network (Intercept)

```
cy.intercept('GET', '/api/users', { fixture: 'u.json' }).as('req')  
cy.intercept('POST', '/api/*', (req) => {  
  req.reply({...}) })  
cy.wait('@req')  
cy.wait('@req').its('response.statusCode').should('eq', 200)  
cy.wait('@req').its('request.body').should('deep.eq', {...})
```

Aliases

```
cy.get('[data-cy="row"]').first().as('firstRow')  
cy.get('@firstRow').click()  
cy.intercept('GET', '/api').as('myReq')  
cy.wait('@myReq')
```

Reading Values

```
el.invoke('text').then((text) => { ... })  
el.invoke('val') // input value  
el.its('length') // array length  
cy.window().its('localStorage')  
Cypress.env('MY_VAR') // env variable
```

Cookies & Storage

```
cy.setCookie('name', 'value')  
cy.getCookie('name').should('have.property', 'value', 'v')  
cy.clearCookies()  
cy.window().then(win =>  
  win.localStorage.getItem('key'))
```

Timers & Clock

```
cy.clock() // take over timers  
cy.tick(3000) // advance 3 seconds  
cy.clock().invoke('restore') // restore real timers
```

Configuration & CLI

```
npx cypress open // interactive mode  
npx cypress run // headless CI  
npx cypress run --browser chrome  
npx cypress run --spec  
  'cypress/e2e/login.cy.js'  
npx cypress run --env  
  apiUrl=https://staging.api.com  
npx cypress run --record --parallel
```

8. Interview Prep Tracker

Use this checklist to track your revision. Print it out or mark it digitally. Aim to review all Easy questions first, then move to Hard, then Scenario-based.

#	Easy Question	1st Review	2nd Review	Confident
1	What is Cypress and how is it different from Selenium?	☐	☐	☐
2	What languages and frameworks does Cypress support?	☐	☐	☐
3	How do you install Cypress?	☐	☐	☐
4	What is the Cypress Test Runner?	☐	☐	☐
5	What is <code>cy.visit()</code> used for?	☐	☐	☐
6	How do you select DOM elements in Cypress?	☐	☐	☐
7	What is the recommended way to select elements for testing?	☐	☐	☐
8	How do you type into an input field in Cypress?	☐	☐	☐
9	How do you click a button in Cypress?	☐	☐	☐
10	What are Cypress assertions and what libraries power them?	☐	☐	☐
11	What is the difference between <code>.should()</code> and <code>.expect()</code> ?	☐	☐	☐
12	What is <code>cy.intercept()</code> ?	☐	☐	☐
13	What are fixtures in Cypress?	☐	☐	☐
14	How do you handle alerts and confirm dialogs in Cypress?	☐	☐	☐
15	What is <code>beforeEach</code> in Cypress?	☐	☐	☐
16	How do you run Cypress in headless mode (CI)?	☐	☐	☐
17	What file structure does Cypress use?	☐	☐	☐
18	What is <code>cy.wrap()</code> ?	☐	☐	☐
19	How do you clear an input field in Cypress?	☐	☐	☐
20	How do you take screenshots and record videos in Cypress?	☐	☐	☐
21	What is <code>cy.wait()</code> and when should you use it?	☐	☐	☐
22	What is the <code>cypress.config.js</code> file?	☐	☐	☐
23	How do you create custom commands in Cypress?	☐	☐	☐
24	What does <code>cy.contains()</code> do?	☐	☐	☐
25	What is <code>cy.reload()</code> used for?	☐	☐	☐
26	What is <code>cy.request()</code> used for?	☐	☐	☐
27	What is the default timeout in Cypress?	☐	☐	☐
28	How do you check if an element exists or doesn't exist?	☐	☐	☐
29	What is the difference between <code>'not.exist'</code> and <code>'not.be.visible'</code> ?	☐	☐	☐
30	How do you check a checkbox in Cypress?	☐	☐	☐

31	How do you select an option from a dropdown?	☐	☐	☐
32	What is cy.exec() used for?	☐	☐	☐
33	How do you hover over an element in Cypress?	☐	☐	☐
34	What is .invoke() used for in Cypress?	☐	☐	☐
35	What is .its() used for?	☐	☐	☐

#	Hard Question	1st Review	2nd Review	Confident
36	How does Cypress handle asynchronous code and the event loop?	☐	☐	☐
37	Why can't you use async/await with Cypress commands directly?	☐	☐	☐
38	What is the difference between cy.stub() and cy.spy()?	☐	☐	☐
39	How do you handle cross-origin iframes in Cypress?	☐	☐	☐
40	What is cy.task() and when would you use it?	☐	☐	☐
41	How do you test file uploads in Cypress?	☐	☐	☐
42	How do you test file downloads in Cypress?	☐	☐	☐
43	How do you set and read cookies in Cypress?	☐	☐	☐
44	What is cy.session() and why is it better than using beforeEach login?	☐	☐	☐
45	How do you perform API-only login in Cypress for speed?	☐	☐	☐
46	How does Cypress retry-ability work?	☐	☐	☐
47	What is the Cypress network layer and how does cy.intercept differ fro...	☐	☐	☐
48	How do you test localStorage and sessionStorage?	☐	☐	☐
49	How do you handle dynamic elements that appear after a delay?	☐	☐	☐
50	How do you run tests in parallel in Cypress?	☐	☐	☐
51	How do you write custom assertions in Cypress with Chai?	☐	☐	☐
52	What is the Cypress Module API and when is it used?	☐	☐	☐
53	How do you handle browser permissions (camera, geolocation) in Cypress...	☐	☐	☐
54	How do you test drag and drop in Cypress?	☐	☐	☐
55	How does Cypress handle shadow DOM?	☐	☐	☐
56	What is the difference between cy.get() and cy.find()?	☐	☐	☐
57	How do you debug Cypress tests?	☐	☐	☐
58	How do you handle multiple tabs or windows in Cypress?	☐	☐	☐
59	How do you configure retries in Cypress for flaky tests?	☐	☐	☐
60	How do you use cy.intercept() to modify a request body before it's sen...	☐	☐	☐
61	What is StaticResponse in cy.intercept()?	☐	☐	☐
62	What is testIsolation in Cypress 12+ and what changed from earlier ver...	☐	☐	☐

63	How do you test WebSocket connections in Cypress?	[]	[]	[]
64	How do you use Cypress Component Testing?	[]	[]	[]
65	How do you measure and assert page performance metrics in Cypress?	[]	[]	[]
66	How do you run Cypress tests with different environment configurations...	[]	[]	[]
67	What is the Cypress grep plugin and how does it help?	[]	[]	[]

#	Scenario Question	1st Review	2nd Review	Confident
66	Scenario: A login page redirects to /dashboard on success. Write a com...	[]	[]	[]
67	Scenario: Test a shopping cart that adds and removes items, updating t...	[]	[]	[]
68	Scenario: A form has conditional fields — Billing address appears only...	[]	[]	[]
69	Scenario: An API call on page load fails with a 500 error. How do you ...	[]	[]	[]
70	Scenario: Test a paginated table that loads more results on clicking N...	[]	[]	[]
71	Scenario: Test that a user cannot submit a form with an empty required...	[]	[]	[]
72	Scenario: Write a test for a search feature with debounce — results ap...	[]	[]	[]
73	Scenario: Test a multi-step wizard form with 3 steps and validation at...	[]	[]	[]
74	Scenario: Your test is flaky — sometimes passes, sometimes fails on .s...	[]	[]	[]
75	Scenario: Test a date picker component — select a date and verify the ...	[]	[]	[]
76	Scenario: You need to test an infinite scroll list. How do you verify ...	[]	[]	[]
77	Scenario: How do you test that an email was actually sent in an E2E te...	[]	[]	[]
78	Scenario: Write a test that verifies a CSV export button triggers a co...	[]	[]	[]
79	Scenario: How do you test behavior that differs by user role (admin vs...	[]	[]	[]
80	Scenario: Test a real-time chat application where messages appear with...	[]	[]	[]
81	Scenario: You have 200 tests and the suite takes 45 minutes. How do yo...	[]	[]	[]
82	Scenario: Test a dropdown that loads options asynchronously from an AP...	[]	[]	[]
83	Scenario: Write a test that verifies only authorized users can access ...	[]	[]	[]
84	Scenario: An image carousel auto-advances every 3 seconds. How do you ...	[]	[]	[]
85	Scenario: Test an autocomplete input that filters suggestions as you t...	[]	[]	[]
86	Scenario: Test a modal dialog — opens, displays correct data, and clos...	[]	[]	[]
87	Scenario: Test that a button is disabled while a form is submitting an...	[]	[]	[]
88	Scenario: Test a feature flag — component renders differently based on...	[]	[]	[]
89	Scenario: Verify a table is sorted correctly when clicking column head...	[]	[]	[]
90	Scenario: Write a reusable Cypress command that seeds a user via API a...	[]	[]	[]
91	Scenario: How do you test a toast notification that appears briefly (2...	[]	[]	[]
92	Scenario: Test that the browser back button restores the previous page...	[]	[]	[]

93	Scenario: Your CI pipeline shows 15 tests consistently failing in CI b...	[]	[]	[]
----	---	-----	-----	-----